Engaging Howto Recipes Slurm

Advanced Job Array

Users often need to submit many jobs to run a program many times with different input parameters or files.

It is straightforward to execute the command `sbatch` in a loop, but this approach is inefficient for job scheduling. When the iteration number is large, it will slow down the Slurm scheduler and affect all users.

A good practice is to use a job array, which is much more efficient. Refer to this page for details of job array.

However, if a user submits too many jobs in a short time, even with a job array, it will still slow down the Slurm scheduler. The maximum number of jobs per user is set to be 500, so that there are not too many jobs queuing in a given time period. A user can submit up to 500 jobs with a job array.

Terminology: job and job array

On this page, the word job means either an individual job or a task in a job array. For example, submitting a job array with 100 tasks means submitting 100 jobs.

When a user needs to run a program more than 500 times, it is recommended to combine serial execution and/or parallel execution with a job array.

On this page, we will introduce serial execution and parallel execution, and how to integrate them with a job array. A simple Python code named mycode.py is used for all examples.

Serial execution

Serial execution means executing multiple programs serially within a job. Here is an example job script that runs 10 programs serially.

```
#!/bin/bash
#SBATCH -t 02:00:00    # tow hours
```

```
#SBATCH -N 1           # 1 node
#SBATCH -n 2           # 2 CPU cores
#SBATCH --mem=2GB      # 2 GB of memory

module load miniforge/24.3.0-0

N_PROGRAMS=10
for i in `seq 1 $N_PROGRAMS`   # Loop for serial execution
do
   python mycode.py  $i    # Run the program serially
done
```

Each program uses 2 CPU cores and 2 GB of memory.

The program is executed 10 times with different input parameters (i.e. the loop index) each time. The next execution will start after the current execution is completed.

Note that the serial execution increases the total run time. Request a wall time that is long enough for all the programs to complete.

Note

This approach is good for programs with short run times. If each program requires a long run time, the total run time may exceed the maximum wall time limit.

Integrate serial execution and job array

To scale up the number of programs, use a job array together with serial execution. Here is an example job script.

```
#!/bin/bash
#SBATCH -t 02:00:00            # Two hours
#SBATCH -N 1                   # 1 node
#SBATCH -n 2                   # 2 CPU cores
#SBATCH --mem=2GB              # 2 GB of memory
#SBATCH --array=0-99           # Job array

module load miniforge/24.3.0-0

nmax=$SLURM_ARRAY_TASK_COUNT    # Num of tasks per array
id=$SLURM_ARRAY_TASK_ID         # Task ID

N_PROGRAMS=10
for i in `seq 1 $N_PROGRAMS`             # Loop for serial execution
do
    index=$(( $nmax * ($i -1) + $id ))   # Calculate the global index
```

```
    python mycode.py $index     # Run the program serially, using the global
index as input
done
```

The number of jobs (tasks) of the array is 100 and each job runs 10 programs serially, so `10 * 100 = 1,000` programs are submitted.

The array task ID (`$SLURM_ARRAY_TASK_ID`) and total number of tasks in the array (`$SLURM_ARRAY_TASK_COUNT`) are used to calculate the global index. Use the global index as the input parameter for the program.

> **Note**
>
> **This approach is useful for submitting a large number of short-run-time programs.**

Also refer to this page for more examples of integrating serial execution and job array.

Parallel execution

Parallel execution means executing multiple programs in parallel within a job. Here is an example job script that runs 10 programs in parallel.

```
#!/bin/bash
#SBATCH -t 00:30:00    # 30 minutes
#SBATCH -N 1           # 1 node
#SBATCH -n 20          # 20 CPU cores
#SBATCH --mem=20GB     # 20 GB of memory

module load miniforge/24.3.0-0

N_PROGRAMS=10
for i in `seq 1 $N_PROGRAMS` # Loop from 1 to number of programs
do
   python mycode.py $i &    # Run a program parallely in background
done
wait   # Wait for all programs to be completed, then exit the batch job.
```

Each program uses 2 CPU cores and 2 GB of memory, so the job requests 20 CPU cores and 20 GB of memory for 10 programs.

Here the main difference from the serial execution is that an `&` mark is added at the end of the execution command, which puts the program in the background, and all 10 programs start to run almost simultaneously.

The `wait` command in the last line ensures that the batch job will not be terminated until all background programs are completed.

> **Note**
>
> **This approach is good when each program requires a small number of CPU cores and a small amount of memory.** If each program requires many CPU cores or large memory, executing multiple programs in parallel would require too many CPUs or too much memory, which may not fit within one node.

Integrate parallel execution and job array

To scale up the number of programs, use a job array together with parallel execution. Here is an example job script.

```
#!/bin/bash
#SBATCH -t 02:00:00             # Two hours
#SBATCH -N 1                    # 1 node
#SBATCH -n 2                    # 2 CPU cores
#SBATCH --mem=2GB               # 2 GB of memory
#SBATCH --array=0-99            # Job array

module load miniforge/24.3.0-0

nmax=$SLURM_ARRAY_TASK_COUNT    # Num of tasks per array
id=$SLURM_ARRAY_TASK_ID         # Task ID

N_PROGRAMS=10
for i in `seq 1 $N_PROGRAMS`    # Loop for serial execution
do
    index=$(( $nmax * ($i - 1) + $id ))   # Calculate the global index
    python mycode.py $index  &   # Run a program parallely in background, using
the global index as input
done
wait     # Wait for all programs to be completed, then exit the batch job.
```

The number of jobs (tasks) of the array is 100 and each job runs 10 programs in parallel, so `10 * 100 = 1,000` programs are submitted.

> **Note**
>
> **This approach is useful to submit a large number of programs, when each program requires small resources (CPUs and memory).**

Integrate sequential execution, parallel execution, and job array

To further scale up the number of programs, one may consider integrating sequential execution, parallel execution, and a job array. Here is an example job script.

```
#!/bin/bash
#SBATCH -t 02:00:00       # Two hours
#SBATCH -N 1              # 1 node
#SBATCH -n 20             # 20 CPU cores
#SBATCH --mem=20GB        # 20 GB of memory
#SBATCH --array=0-99      # Job array

module load miniforge/24.3.0-0

nmax=$SLURM_ARRAY_TASK_COUNT     # Num of tasks per array
id=$SLURM_ARRAY_TASK_ID          # Task ID

N_SERIAL=10
N_PARALLEL=10
for i in `seq 1 $N_SERIAL`        # Loop for serial executions
do
   for j in `seq 1 $N_PARALLEL`   # Loop for parallel executions
   do
     index=$(( $nmax * ( $N_PARALLEL * ($i-1) + ($j-1) ) + $id ))  # Global index
     python mycode.py $index &    # Run a program in the background, using the
global index as input.
   done
   wait    # Wait for all parallel executions to complete, then go to the next
iteration in the loop of serial executions.
done
```

The number of jobs (tasks) of the array is 100 and each job runs `10 * 10 = 100` programs in the nested loops of serial and parallel executions, so `100 * 100 = 10,000` programs are submitted.

> **Note**
>
> **This approach is useful for submitting a large number of programs, when each program requires a short run time and small resources (CPUs and memory).**

August 13, 2025